

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 2
6ta práctica (tipo b)
Primer Semestre 2026

Indicaciones Generales:

- Duración: 110 minutos.

NO SE PERMITE EL USO DE APUNTES DE CLASE, FOTOCOPIAS NI MATERIAL IMPRESO

- No se pueden emplear **variables globales**, **NI OBJETOS** (con excepción de los elementos de `iostream`, `omanip` y `fstream`). **NO PUEDE UTILIZAR LA CLASE `string`**. Tampoco se podrán emplear las funciones `malloc`, `realloc`, `memset`, `strtok` o `strdup`, igualmente no se puede emplear cualquier función contenida en las bibliotecas `stdio.h`, `cstdio` o similares y que puedan estar también definidas en otras bibliotecas. **NO PODRÁ EMPLEAR PLANTILLAS EN ESTE LABORATORIO**
- Deberá modular correctamente el proyecto en archivos independientes. LAS SOLUCIONES DEBERÁN DESARROLLARSE BAJO UN ERICTO DISEÑO DESCENDENTE. Cada función **NO debe sobrepasar las 20 líneas de código aproximadamente**. El archivo `main.cpp` solo podrá contener la función `main` de cada proyecto y el código contenido en él solo podrá estar conformado por tareas implementadas como funciones. En el archivo `main.cpp` deberá colocar un comentario en el que coloque claramente su nombre y código, **de no hacerlo se le descontará 0.5 puntos en la nota final**.
- El código comentado **NO SE CALIFICARÁ**. De igual manera **NO SE CALIFICARÁ** el código de una función si esta función no es llamada en ninguna parte del proyecto o su llamado está comentado.
- Los programas que presenten errores de sintaxis o de concepto se calificarán en base al 40% de puntaje de la pregunta. Los que no muestren resultados o que estos no sean coherentes en base al 60%.
- Se tomará en cuenta en la calificación el uso de comentarios relevantes.
- **TAMPOCO SE PODRÁ EMPLEAR LA CLÁUSULA `protected` NI LA CLÁUSULA `friend`, DE HACERLO NO SE LE CALIFICARÁN LAS CLASES INVOLUCRADAS.**

SE LES RECUERDA QUE, DE ACUERDO AL REGLAMENTO DISCIPLINARIO DE NUESTRA INSTITUCIÓN, CONSTITUYE UNA FALTA GRAVE COPIAR DEL TRABAJO REALIZADO POR OTRA PERSONA O COMETER PLAGIO.

NO SE HARÁN EXCEPCIONES ANTE CUALQUIER TRASGRESIÓN DE LAS INDICACIONES DADAS EN LA PRUEBA

- Puntaje total: 20 puntos.

INDICACIONES INICIALES

Cree un proyecto de C++ en CLion siguiendo estrictamente las indicaciones que a continuación se detallan:

- La unidad de trabajo será **t:** (Si lo coloca en otra unidad, no se calificará su laboratorio y se le asignará como nota cero)
- Cree allí una carpeta con el nombre **"CO_PA_PN_Lab06_2026_1"** donde **CO** indica: Código del alumno, **PA** indica: Primer Apellido del alumno y **PN** primer nombre (de no colocar este requerimiento se le descontará 3 puntos de la nota final). **Allí colocará el proyecto solicitado en la prueba.**

Cuestionario:

La finalidad principal de este laboratorio es la de reforzar los conceptos contenidos en el capítulo 7: "Herencia".

Descripción del Problema:

Se le ha solicitado desarrollar un sistema en C++ para gestionar las citas de una veterinaria:

PARTE 01 (8 puntos): CREACIÓN DE LAS CLASES

Se solicita que desarrolle un proyecto **"LABORATORIO06"** dentro de la carpeta correspondiente, **DE NO COLOCAR ESTE REQUERIMIENTO SE LE DESCONTARÁ 2 PUNTOS DE LA NOTA FINAL**, en la cual se declaren las clases descritas con las relaciones necesarias, que permitan manipularlas empleando herencia:

- ❖ **Para manejar a las mascotas:** La clase se denominará "**Mascota**" y deberá contener lo siguiente: 1) un atributo denominado **codigo** (**int**) donde se almacena el código de la mascota, 2) un atributo denominado **nombre** (**char***), 3) un atributo denominado **tipo** (**char**) donde se guarda el tipo de mascota A(Ave), F(Felino), C(Canino) y 4) un atributo denominado **raza** (**char***).
- ❖ **Para manejar las citas:** La clase se denominará "**Cita**" y deberá contener lo siguiente: 1) un atributo denominado **fecha** (**int**), 2) un atributo denominado **hora** (**int**) y 3) un atributo denominado **Amascota** (**class Mascota**) donde se guardará la información de la mascota.
- ❖ **Para manejar las citas de control:** La clase se denominará "**Control**" y deberá contener lo siguiente: 1) un atributo denominado **costo** (**double**) y 2) un atributo denominado **codmed** (**int**) que guardará el código del médico. Además, esta clase posee datos heredados de la clase **Cita**.
- ❖ **Para manejar las citas de operación:** La clase se denominará "**Operacion**" y deberá contener lo siguiente: 1) un atributo denominado **anestesiageneral** (**bool**), donde se almacena si la operación requiere anestesia general, si no requiere anestesia general se considera anestesia parcial, 2) un atributo denominado **nummedicos** (**int**), donde se guardará la cantidad de médicos que participarán en la operación y 3) un atributo denominado **total** (**double**), donde se guardará el costo total de la operación. Además, esta clase posee datos heredados de la clase **Cita**.
- ❖ **Para manejar las citas de vacunación:** La clase se denominará "**Vacuna**" y deberá contener lo siguiente: 1) un atributo denominado **dosís** (**int**) y 2) un atributo denominado **meses** (**int**), donde se guardará después de cuantos meses se debe aplicar la siguiente dosis. Además, esta clase posee datos heredados de la clase **Cita**.
- ❖ **Para manejar todas las citas:** La clase se denominará "**Veterinaria**" y deberá contener lo siguiente: 1) un atributo denominado **arrControl**, este atributo es un arreglo dinámico de la clase **Control**, donde se guardarán todas las citas por control, 2) un atributo denominado **arrVacuna**, este atributo es un arreglo estático de la clase **Vacuna**, 3) un atributo denominado **arrOperacion**, este atributo es un arreglo estático de la clase **Operacion**, donde se guardarán todas las citas por operación, 4) un atributo **numdatControl** (**int**), donde se guardará la cantidad de datos del **arrControl** y 5) un atributo **capaControl** (**int**), donde se guardará la capacidad de datos del **arrControl**, si emplea memoria dinámica incremental en el llenado del arreglo.

"DEBE EMPLEAR OBLIGATORIAMENTE LOS NOMBRES DE LAS CLASES Y SUS ATRIBUTOS"

PARTE 02 (12 puntos): PROCESAMIENTO DE DATOS

Con las clases indicas debe realizar las siguientes operaciones:

- (8 puntos) En la clase **Veterinaria** debe implementar el método **cargacitas**, que se encargara de la lectura del archivo "atenciones.csv", con la finalidad de llenar los arreglos según el tipo de cita que corresponda.

atenciones.csv									
C	8/04/2026	10	103	Rocky	C	Bulldog	5689	20	
O	9/04/2026	11	105	Toby	C	Beagle	100	0	1
V	9/04/2026	13	106	Simba	F	Comun	3	3	
...									
Tipo_de_cita, fecha, hora, cód_de_mascota, nombre, tipo, raza, médico/total/dosis, monto/anestesia/meses, núm. de médicos									

- (4 puntos) En la clase **Veterinaria** implementar el método **muestracitas**, que se encargue de realizar la impresión de un archivo de reporte (**sin usar el carácter '\t'**), que muestre el contenido de los tres arreglos de citas.

Citas de Control:					
Fecha	Hora	Raza	Nombre	Cod. Medico	Monto
2025.12.20	13:00	Americano	Dali	1222	15.00
2026.04.10	09:00	Ninfa	Fenix	8921	60.00
2025.04.07	09:00	Canario	Axl Rose	8921	60.00
2025.12.20	10:00	Canario	Debbie Harry	3572	60.00

2025.11.03	11:00	Agapornis	Pimpona	9331	60.00
...					
Monto por Control: 865.00					
Citas de Vacunas:					
Fecha	Hora	Raza	Nombre	Dosis	Meses próxima Dosis

2026.04.18	13:00	Canario	Slash	1	12
2026.04.14	12:00	Canario	Jon Bon Jovi	1	12
2026.04.18	10:00	Labrador	Luna	3	1
...					
Numero de vacunas: 13					
Citas de Operaciones:					
Fecha	Hora	Raza	Nombre	Anestesia	Numero de medicos

2026.04.09	11:00	Beagle	Toby	Parcial	1
2026.04.25	10:00	Siberiano	LuisXV	General	2
2026.03.12	13:00	Americano	Dali	Parcial	1
2026.04.19	11:00	Beagle	Toby	General	2
Total por operaciones: 570.00					
...					

Consideraciones:

Para el desarrollo debe considerar el siguiente código:

```
#include "Biblioteca/Veterinaria.h"

int main() {
    Veterinaria vet;

    vet.cargacitas();
    vet.muestracitas();

    return 0;
}
```

**NO PUEDE
CAMBIAR
ESTE CÓDIGO**

Recuerde que si no usa herencia la respuesta no será válida.

- No debe repetir código
- La gestión de los datos debe ser considerar el encapsulamiento en todo momento, en otras palabras, los atributos y métodos deben definirse donde le corresponde. Por ejemplo, si accede a datos de la clase base deben definirse métodos en dicha clase no en la derivada
- Para el arreglo dinámico **arrControl** puede emplear memoria dinámica exacta o incremental. Si usa incremental, los incrementos deben ser de 5 en 5.
- Debe desarrollar los setters y getters de todos los atributos de las clases
- Debe desarrollar los constructores por defecto y destructores de todas las clases

Al finalizar la práctica, comprima la carpeta de su proyecto empleando el programa Zip que viene por defecto en el Windows, **no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares.**

Profesores del curso:
Rony Cueva
Erick Huiza
Miguel Guanira

Erasmus Gómez
Ana Roncal

San Miguel, 05 de junio del 2026.